

第三次作业反馈

林洛昊

2024.9

1 相关知识回顾

这次的课堂和作业主要是链表相关的内容，链表是考试的重难点，请大家一定要掌握这部分的知识，我在此列举一下 ppt 上的相关内容

首先是定义：

```
typedef int ElemType; // 元素类型
typedef struct LNode {
    ElemType data; // 数据域
    LNode *next;   // 指针域
} *LinkList;
```

然后是 ppt 上给出的一些基本操作的算法实现，这些函数大家在写作业的时候可以放心用：

```
void InitList_L(LinkList &L) {
    L=NULL;
}

int ListLength_L(LinkList L) {
    LNode *p=L; int j=0;
    while (p) {
        p=p->next; ++j;
    }
    return j;
}

int LocateElem_L(LinkList L, ElemType e) {
    LNode *p=L; int j=1;
    while (p && p->data != e) {
        p=p->next; ++j;
    }
    return p ? j : 0; //if(p) return j;else{return 0};
}

LNode* LocateElem_L2(LinkList L, ElemType e) {
    LNode *p=L;
    while (p && p->data != e) p=p->next;
    return p;
}
```

```

}

void ListInsert_L(LinkList &L, int i, ElemType e) {
    LNode *s=new LNode; s->data=e;
    if (i == 1) { s->next=L; L=s; }
    else {
        LNode *q=L; int j=1;
        while (q && j < i-1) { q=q->next; ++j; }
        if (!q || j > i-1) ErrorMsg("Invalid i value");
        s->next=q->next;
        q->next=s;
    }
}

void ListInsert_L(LinkList &L, LNode *p, LNode *s) {
    if (p == L) {
        s->next=L;
        L=s;
    }
    else {
        LNode *q=L;
        while (q && q->next != p) q=q->next;
        if (!q) ErrorMsg("Invalid p value");
        s->next=p; q->next=s;
    }
}

void ListDelete_L(LinkList &L, LNode *p, ElemType &e) {
    if (p == L) L=L->next;
    else {
        LNode *q=L;
        while (q->next && q->next != p) q=q->next;
        if (!(q->next)) ErrorMsg("Invalid p value");
        q->next=p->next;
    }
    e=p->data; delete p;
}

```

2 作业题目解答

2.4 已知单链表 La 中数据元素按非递减有序排列，按两种不同情况，分别写出算法，将元素 x 插到 La 的合适位置上，保持该表的有序性：(1)La 带头结点 (2)La 不带头结点

(1) 带头结点的情况

```

void ListInsert_sq(Linklist &La, ElemType e) {
    LNode *p=La,*s=new LNode;
    s->data=e;

```

```

    while((p->next != NULL)&&(p->next->data<e)) p=p->next;
    s->next=p->next;
    p->next=s;
}

```

(2) 不带头结点的情况

```

void ListInsert_sq(Linklist &La, ElemType e) {
    LNode *p=La,*s=new LNode;
    s->data=e;
    if(p->data>e){
        s->next=La;
        La=s;
    }
    else{
        while((p->next != NULL)&&(p->next->data<e)) p=p->next;
        s->next=p->next;
        p->next=s;
    }
}

```

不带头结点时，我们需要多考虑一个情况，那就是插入的元素比第一个元素小的时候，需要把新插入的元素放在原本的头结点前面，并更新头结点，大家写代码的时候也要注意这种边界情况噢

2.8' 已知链表用顺序存储结构表示，表中数据元素为 n 个正整数，试写一个算法，分离该表中的奇数和偶数，使得所有奇数集中放在左侧，偶数集中放在右侧，要求不借用辅助数组且时间复杂度为 $O(n)$

在上一次作业的 2.8 中，我们使用了双哨兵左右夹击的方法，但由于链表的特性，我们这次无法使用左右夹击的方法了，不过幸运的是，由于链表的 next 指针修改起来比较方便，所以我们可以通过遍历链表的方式，用奇数尾和偶数尾来指向并串起奇数结点和偶数结点，最后再用奇数尾->next 来指向偶数头。顺带一提，我给的是有头结点的写法，如果没有头结点的话，这种写法会出问题，大家可以想想为什么。

```

void oddEvenList(Linklist &L) {
    if (!L || !L->next) return;

    LNode* oddHead=L;
    LNode* evenHead=L->next;
    LNode* oddTail=oddHead;
    LNode* evenTail=evenHead;

    LNode* curr=evenHead->next;
    while (curr) {
        if (curr->val%2 == 1) {
            oddTail->next=curr;
            oddTail=oddTail->next;
        }
        else {
            evenTail->next=curr;
            evenTail=evenTail->next;
        }
    }
}

```

```

        curr=curr->next;
    }

    oddTail->next=evenHead;
    evenTail->next=NULL;
}

```

细心的同学可以留意到，如果没有头结点，且第一个结点为偶数的话，我这个写法会导致 oddHead 指向第一个结点也就是偶数，导致出现错误，如果没有头结点的话，可以考虑改装成我下面这种写法：

```

void oddEvenList(Linklist& L) {
    if (!L || !L->next) return;

    LNode *oddHead=NULL;
    LNode *oddTail=NULL;
    LNode *evenHead=NULL;
    LNode *evenTail=NULL;
    LNode *curr=L;

    while (curr) {
        if (curr->val % 2 == 1) {
            if (!oddHead) {
                oddHead=curr;
                oddTail=curr;
            } else {
                oddTail->next=curr;
                oddTail=oddTail->next;
            }
        } else {
            if (!evenHead) {
                evenHead=curr;
                evenTail=curr;
            } else {
                evenTail->next=curr;
                evenTail=evenTail->next;
            }
        }
        curr=curr->next;
    }

    if (oddHead) {
        oddTail->next=evenHead;
        if (evenTail) {
            evenTail->next=NULL;
        }
        L=oddHead;
    } else {
        L=evenHead;
    }
}

```

```
}
```

3 补充阅读资料

Hello 算法-第四章数组与链表

4 总结

链表是本学期课程的一个重难点，在考试和大作业中出现的频率会比较高，大家一定要掌握其相关知识。线性表和链表是后续章节的基础，大家如果在学习后续章节时遇到困难，记得复习一下线性表和链表的相关知识，防止自己弄错了一些基本概念。